What is etcd?

Professional Answer

etcd is a distributed, highly available key-value store used to persist the entire cluster state, including configuration, metadata, and secrets.

Unprofessional Answer

etcd stores Kubernetes data.

What is etcd in Kubernetes architecture?

etcd is a distributed and consistent key-value store used to store the entire cluster state.
It acts as the single source of truth for configuration, metadata, and secrets.
etcd uses the Raft consensus algorithm to ensure strong consistency.
Only the kube-apiserver communicates directly with etcd.
If etcd data is lost, the Kubernetes cluster cannot be recovered.

Why is etcd critical to Kubernetes?

Professional Answer

etcd is the source of truth for the cluster; losing etcd means losing cluster state, configurations, and workloads.

Unprofessional Answer

Without etcd, Kubernetes won't work.

2. etcd (Cluster Memory Internals)

What it is internally

- Distributed key-value store
- Uses **RAFT consensus**
- Strong consistency (CP system)

Internal Working

- Stores everything as key-value pairs
- Example:
- /registry/pods/default/nginx

Key Internals

1. **Leader Election**
    - One leader
    - Followers replicate data
2. **Write Path**
    - Request → Leader → Majority acknowledgment → Commit
3. **Watch Mechanism**
    - API server and controllers watch etcd for changes
    - Enables event-driven architecture

Why etcd is critical

- Lose etcd → lose cluster state
- Backups are mandatory in production

> Alright, let's deep-dive **etcd** like a **senior DevOps / Kubernetes architect** would explain it in interviews.
> I'll keep it **exam-ready**, **production-oriented**, and **interview-sharp**.
>
> ---
>
> **etcd – Kubernetes Brain (Interview Master Guide)**
>
> **1. What is etcd in Kubernetes?**
>
> **Answer:**
> etcd is a **distributed, consistent, and highly available key-value store** used by Kubernetes to store **all cluster state**.
> It acts as the **single source of truth** for Kubernetes.
> Every object—Pods, Nodes, Secrets, ConfigMaps, Deployments—is stored in etcd.
> If etcd is lost, the cluster loses its state.
> Kubernetes control plane components continuously read from and write to etcd.

2. Why Kubernetes uses etcd?

Answer:
Kubernetes needs **strong consistency**, not eventual consistency.
etcd provides **linearizable reads and writes** using the **Raft consensus algorithm**.
It ensures all control plane components see the same cluster state.
etcd supports **watch functionality**, enabling real-time updates.
This makes scheduling, reconciliation, and self-healing possible.

---

3. Where does etcd run?

Answer:
In production clusters, etcd runs as **static pods** on control plane nodes.

In managed Kubernetes (EKS/GKE/AKS), etcd is fully managed by the cloud provider.
It typically listens on ports **2379 (client)** and **2380 (peer)**.
etcd is isolated from worker nodes for security reasons.
Only control plane components can access it.

4. What data is stored in etcd?

Answer:
etcd stores **cluster metadata**, not application data.
Examples include Pods, ReplicaSets, Services, Secrets, RBAC policies, and Node info.
Secrets are stored **base64-encoded**, not encrypted by default.
Encryption at rest must be explicitly enabled.
etcd does not store container logs or runtime data.

---

5. How control plane components interact with etcd

Answer:

- **API Server** is the **only component** that directly talks to etcd
- **Scheduler** reads pod specs via API Server
- **Controller Manager** watches state changes via API Server
- **kubelet** never talks to etcd directly
  This design enforces security and consistency.

6. What is Raft consensus in etcd?

Answer:
Raft ensures all etcd members agree on the same data.
One node acts as the **leader**, others as **followers**.
Writes go through the leader and are replicated to followers.
A quorum (majority) is required for any write.
This prevents split-brain scenarios.

---

7. Why etcd cluster must have odd number of nodes?

Answer:
Odd numbers ensure **quorum availability**.
3-node cluster tolerates 1 failure.
5-node cluster tolerates 2 failures.
Even numbers don't increase fault tolerance.
This is a fundamental Raft requirement.

8. What happens if etcd goes down?

Answer:

- No new Pods can be scheduled
- kubectl commands fail
- Existing workloads continue running
- Controllers cannot reconcile state
- Cluster becomes **read-only or completely unusable**

---

9. How does Kubernetes detect etcd failure?

Answer:
API Server health checks fail.
Control plane components start timing out.
kubectl commands return connection errors.
Leader election may fail.
Cluster events stop updating.

10. How to check etcd health?

ETCDCTL_API=3 etcdctl endpoint health

Answer:
This checks if all etcd members are reachable.
Unhealthy members indicate quorum risk.
Always run from control plane node.
Use TLS certificates in production.
Never expose etcd publicly.

---

11. How do you back up etcd?

ETCDCTL_API=3 etcdctl snapshot save snapshot.db

Answer:
Snapshots capture full cluster state.
Backups must be taken **frequently**.
Store snapshots off the cluster.
Backups are mandatory before upgrades.
This is the only way to recover a dead cluster.

12. How do you restore etcd?

etcdctl snapshot restore snapshot.db

Answer:
Restore creates a **new etcd data directory**.
All control plane components must be stopped.
After restore, API Server reconnects to new data.

Cluster resumes from snapshot state.
Node re-registration may be required.

13. How is etcd secured?

Answer:
etcd uses **mTLS authentication**.
Client certs authenticate API Server.
Peer certs authenticate etcd members.
Encryption at rest protects sensitive data.
Network access must be strictly limited.

---

14. Common etcd performance bottlenecks

Answer:

- Slow disks (etcd is disk-intensive)
- High write frequency
- Large number of watches
- Frequent leader elections
- Network latency between members

15. Why SSDs are mandatory for etcd?

Answer:
etcd writes to disk synchronously.
Latency directly impacts API Server response time.
Slow disks cause cluster slowness.
HDDs increase risk of timeouts.
SSDs are a production requirement.

---

16. What is etcd compaction?

Answer:
etcd keeps historical versions of data.
Compaction removes old revisions.
Reduces disk usage.
Improves performance.
Must be scheduled regularly.

17. What is etcd defragmentation?

Answer:
Even after deletion, disk space is not freed.

Defragmentation reclaims unused space.
Should be done during low traffic.
Improves IO performance.
Critical for long-running clusters.

---

18. What happens if etcd quorum is lost?

Answer:
Writes stop completely.
Leader cannot be elected.
Cluster becomes unavailable.
Manual intervention is required.
Restoring quorum is top priority.

19. Production failure scenario

Question: API Server is slow, Pods scheduling delayed
Answer:
Check etcd disk latency.
Inspect etcd leader stability.
Verify snapshot/compaction status.
Check resource usage on control plane nodes.
etcd is often the root cause.

---

20. Ultra-hard interview question

Question:
“Pods are running, but kubectl hangs. What’s wrong?”

Answer:
etcd is likely unhealthy.
Existing workloads don’t need etcd to run.
But control plane operations do.
Check etcd quorum and disk IO.

21. Professional vs Unprofessional answer

Unprofessional:

etcd stores Kubernetes data.

Professional:

etcd is a strongly consistent distributed key-value store that acts as the authoritative state store for Kubernetes, enabling reconciliation, leader election, and cluster self-healing using the Raft consensus algorithm.

---

22. One-line exam answer

etcd is the highly available, consistent datastore that stores the entire Kubernetes cluster state and enables control plane decision-making.

Perfect topic 😈 — this is **real-world, senior-level Kubernetes knowledge**.
Below are **actual-style production outages caused by etcd**, explained the way **postmortems + interviews** expect.

🔥 **Real Production Outages Caused by etcd (Deep Dive)**

1. Disk Latency Spike → Entire Cluster Freezes

What happened

- etcd disk was on **slow or saturated storage**
- fsync latency crossed 100–200ms
- etcd leader could not commit writes fast enough

Symptoms

- kubectl get pods hangs
- New Pods stuck in Pending
- Controllers stop reconciling
- Existing Pods keep running

Root Cause

etcd is **synchronous-write heavy**
Slow disks = slow consensus = slow API Server

How it was fixed

- Moved etcd to SSD / faster IOPS
- Separated etcd from noisy workloads
- Added disk monitoring alerts

Interview takeaway

"If kubectl hangs but Pods run, suspect etcd disk latency."

2. etcd Disk Full → API Server Down

What happened

- etcd disk reached **100% usage**
- Compaction + defragmentation not configured

Symptoms

- API Server crash looping
- etcdserver: mvcc: database space exceeded
- No kubectl access
- Deployments stuck

Root Cause

etcd stores historical revisions
Deletes don't free disk without defrag

Fix

- Emergency etcd defrag
- Increased disk size
- Enabled auto-compaction

Interview takeaway

etcd disk fills **even if Pods are deleted**

3. Loss of Quorum → Total Control Plane Outage

What happened

- 3-node etcd cluster
- 2 nodes went down (maintenance + failure)

Symptoms

- No leader election
- Writes blocked
- API Server returns 500

Root Cause

Raft requires **majority quorum**
3-node cluster needs **2 nodes alive**

Fix

- Restore failed node
- Avoid simultaneous control-plane maintenance
- Use PodDisruptionBudgets

Interview takeaway

Losing quorum = cluster brain-dead

4. Clock Skew Between etcd Nodes

What happened

- NTP misconfigured
- Time drift between etcd members

Symptoms

- Frequent leader elections
- API Server timeouts
- Random scheduling failures

Root Cause

Raft is time-sensitive
Clock skew breaks heartbeat timings

Fix

- Enforced NTP sync
- Restarted etcd members

Interview takeaway

etcd + bad clocks = chaos

5. etcd Backup Missing → Irrecoverable Cluster Loss

What happened

- etcd data corrupted
- No recent snapshots available

Symptoms

- Cluster refused to start
- All Kubernetes metadata lost

Root Cause

No etcd backups
etcd = single source of truth

Fix

- Cluster rebuilt from scratch
- Applications redeployed manually

Interview takeaway

No etcd backup = no cluster

6. Large Secrets & ConfigMaps Explosion

What happened

- Teams stored large cert bundles in Secrets
- Thousands of large objects

Symptoms

- etcd memory spikes
- API Server slow
- Watch events lagging

Root Cause

etcd is **not a blob store**
Large objects increase watch overhead

Fix

- External secret stores (Vault, AWS Secrets Manager)
- Reduced object size

Interview takeaway

etcd is metadata storage, not file storage

7. etcd Leader on Weak Node

What happened

- Leader elected on underpowered control-plane node

Symptoms

- High latency
- Frequent leader re-elections

Root Cause

Leader handles **all writes**
Weak node = bottleneck

Fix

- Uniform hardware
- Resource isolation for etcd

Interview takeaway

etcd leader performance matters more than followers

8. Network Partition Between etcd Members

What happened

- Firewall rule blocked peer port 2380

Symptoms

- Split cluster
- Leader keeps changing
- Writes failing intermittently

Root Cause

etcd peer communication broken

Fix

- Fixed network rules
- Verified peer TLS connectivity

Interview takeaway

etcd needs reliable low-latency network

9. API Server Overload → etcd Overload

What happened

- Massive kubectl get + CI jobs
- High watch/list requests

Symptoms

- etcd CPU spikes
- Slow cluster operations

Root Cause

API Server floods etcd with reads
No rate limiting

Fix

- API Server request throttling
- Reduced polling scripts

Interview takeaway

Interview takeaway

etcd can be DoS'ed internally

10. Upgrade Gone Wrong → etcd Data Incompatible

What happened

- etcd version mismatch during upgrade
- Incorrect upgrade order

Symptoms

- etcd failed to start
- Control plane stuck

Root Cause

Skipped etcd upgrade compatibility matrix

Interview takeaway

etcd upgrades are not casual

🔥 Ultra-Hard Interview Questions (Real)

Q1:

Pods are running but scaling doesn't work. Why?

Answer:
etcd is unhealthy.
Scaling requires writing desired state.
Existing Pods don't need etcd.

Q2:

How can etcd bring down a "healthy" cluster?

Answer:
Control plane depends entirely on etcd.
Without etcd writes, reconciliation stops.
Cluster appears alive but is frozen.

Q3:

What metric would you monitor first for etcd?

Answer:
Disk latency (fsync duration).
Then leader changes.
Then database size.

🔥 Golden Rule (Senior-Level)

Most "Kubernetes issues" are actually etcd issues wearing a disguise.